Lecture 28.08: Common Pruning Rules & Hyperparameters

📌 Topic Overview

In this lecture, we examine the official Scikit-Learn documentation for `DecisionTreeClassifier` and understand each hyperparameter used to prevent overfitting and control tree size.

🛠️ Key Pruning Hyperparameters in Scikit-Learn

1. `max_depth` (int, default=None)

- **Meaning**: The maximum allowed depth (vertical distance from root to leaf) of the tree.
- **Why it matters**: If left as `None`, nodes expand until all leaves are pure or contain fewer than `min_samples_split` samples. Setting `max_depth=3` or `4` is the simplest and most effective pre-pruning rule.

2. `min_samples_split` (int or float, default=2)

- **Meaning**: The minimum number of samples a node must contain before it is allowed to split.
- **Why it matters**: If a node has only 3 samples and `min_samples_split=10`, the tree will not split it further. It forces the tree to stop when subsets become too small.

3. `min_samples_leaf` (int or float, default=1)

- **Meaning**: The minimum number of samples that every terminal leaf node must possess.
- **Why it matters**: A split will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This directly prevents isolating single outlier samples (like the $[14y, 1n]$ case from the previous lecture!).

4. `max_leaf_nodes` (int, default=None)

- **Meaning**: Grow a tree with at most `max_leaf_nodes` in best-first fashion.
- **Why it matters**: Places a hard ceiling on the total complexity of the tree.

5. `class_weight` (dict or "balanced", default=None)

- **Meaning**: Weights associated with classes.

- **Why it matters**: When working with imbalanced data (e.g., 90% No, 10% Yes), setting `class_weight="balanced"` automatically adjusts weights inversely proportional to class frequencies: $w_c = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_c}$.

6. `ccp_alpha` (non-negative float, default=0.0)

- **Meaning**: Complexity parameter used for Minimal Cost-Complexity Pruning (post-pruning).
- **Why it matters**: By default, `ccp_alpha=0.0` (no post-pruning). Increasing α prunes away the weakest subtrees.

💻 Python Implementation

```
from sklearn.tree import DecisionTreeClassifier

def create_regularized_classifier(max_depth=4, min_samples_split=10, min_samples_l
    """
    Builds a regularized DecisionTreeClassifier configured with key pre-pruning
    hyperparameters to prevent tree overfitting.
    """
    model = DecisionTreeClassifier(
        criterion='gini',
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf,
        class_weight='balanced',
        random_state=42
    )
    return model

# --- Function Call & Inspection ---
tree_clf = create_regularized_classifier()
print("Initialized Regularized DecisionTreeClassifier:")
print(f"  - max_depth:         {tree_clf.max_depth}")
print(f"  - min_samples_split: {tree_clf.min_samples_split}")
print(f"  - min_samples_leaf:  {tree_clf.min_samples_leaf}")
print(f"  - class_weight:      {tree_clf.class_weight}")
```

🔑 Key Takeaways

1. `max_depth`, `min_samples_split`, and `min_samples_leaf` are the three primary pre-pruning knobs.
2. `class_weight="balanced"` prevents majority-class bias in imbalanced datasets.

3. `ccp_alpha` controls Minimal Cost-Complexity post-pruning.